Gestion des utilisateurs et des privilèges

La gestion des droits dans PostgreSQL repose sur le concept de **rôles**, qui unifie les notions d'utilisateurs et de groupes.

1. Différence entre rôles et utilisateurs

- **Utilisateur (User) : C'est un rôle qui possède l'attribut LOGIN. Il peut se connecter à la base de données avec un identifiant et un mot de passe.**
- **Rôle (au sens strict) : Souvent utilisé comme un groupe. C'est un rôle qui n'a généralement pas l'attribut LOGIN. Il sert de conteneur pour des privilèges que l'on distribue ensuite à des "utilisateurs"**

2. Gestion des Rôles (Utilisateurs et Groupes)

Un rôle peut se connecter s'il possède l'attribut LOGIN.

- **Créer un utilisateur :** CREATE ROLE nom_utilisateur WITH LOGIN PASSWORD 'mon_mot_de_passe';.
- **Créer un groupe :** CREATE ROLE nom_groupe; (par défaut, sans LOGIN).
- **Ajouter un membre à un groupe :** GRANT nom_groupe TO nom_utilisateur;.

3. Attribution des Privilèges (GRANT)

Les privilèges contrôlent les actions sur les objets (bases, schémas, tables).

- **Connexion à une base :** GRANT CONNECT ON DATABASE ma_base TO mon_role;.
- **Usage d'un schéma :** GRANT USAGE ON SCHEMA mon_schema TO mon_role;.
- **Actions sur une table :** GRANT SELECT, INSERT, UPDATE ON TABLE ma_table TO mon_role;.
- **Tout accorder :** GRANT ALL PRIVILEGES ON TABLE ma_table TO mon_role;.

4. Révocation des Droits (REVOKE)

Pour retirer un privilège précédemment accordé :

- **Syntaxe :** REVOKE SELECT ON TABLE ma_table FROM mon_role;.
- **Retirer tous les droits :** REVOKE ALL ON TABLE ma_table FROM mon_role;.

5. Modification d'un utilisateur (ALTER ROLE / ALTER USER)

Dans PostgreSQL, les termes USER et ROLE sont quasiment interchangeables (un utilisateur est techniquement un rôle avec le droit de connexion).

- **Changer le mot de passe :**

ALTER USER nom_utilisateur WITH PASSWORD 'nouveau_mot_de_passe';

- **Renommer un utilisateur :**

ALTER USER ancien_nom RENAME TO nouveau_nom;

- **Ajouter des privilèges (ex: devenir superutilisateur) :**

ALTER USER nom_utilisateur SUPERUSER;

(Utilisez NOSUPERUSER pour retirer ce droit).

- **Définir une date d'expiration pour le compte :**

ALTER USER nom_utilisateur VALID UNTIL '2026-12-31 23:59:59';

6. Suppression d'un utilisateur (DROP USER / DROP ROLE)

La commande de base est simple, mais elle échouera si l'utilisateur possède encore des objets (tables, schémas) ou des privilèges dans une base de données.

- **Commande standard :**

DROP USER nom_utilisateur;

- **Suppression conditionnelle (évite l'erreur si l'utilisateur n'existe pas) :**

DROP USER IF EXISTS nom_utilisateur;

7. Gestion des dépendances avant suppression

Si vous recevez une erreur indiquant que l'utilisateur "possède des objets", vous devez soit les supprimer, soit réattribuer leur propriété à un autre utilisateur avant de pouvoir lancer le DROP.

- **Réattribuer tous les objets à un autre utilisateur :**

REASSIGN OWNED BY nom_utilisateur TO administrateur;

- **Supprimer tous les objets appartenant à l'utilisateur :**

DROP OWNED BY nom_utilisateur;

- **Une fois ces actions effectuées, vous pouvez supprimer l'utilisateur :**

DROP USER nom_utilisateur;

8. Bonnes Pratiques de Sécurité

- **Principe du moindre privilège :** n'accordez que le strict nécessaire.
- **Utiliser des rôles de groupe :** gérez les droits au niveau du groupe et non par utilisateur individuel pour simplifier l'administration.
- **Sécuriser le schéma public :** révoquez le droit de création par défaut (REVOKE CREATE ON SCHEMA public FROM PUBLIC;) pour éviter que n'importe quel utilisateur puisse y créer des objets.
- **Privilèges par défaut :** utilisez ALTER DEFAULT PRIVILEGES pour définir automatiquement les droits sur les futurs objets créés.

Scripte pour une hiérarchie de sécurité

Voici un script SQL structuré pour mettre en place une hiérarchie propre sur une base de données PostgreSQL, en séparant les accès **Lecture Seule (RO)** et **Lecture-Écriture (RW)**.

```
-- 1. Nettoyage et Sécurisation (Optionnel)

-- Révoquer les droits de création publics pour éviter le désordre

REVOKE CREATE ON SCHEMA public FROM PUBLIC;

-- 2. Création des rôles de groupe (sans LOGIN)

CREATE ROLE group_read_only;

CREATE ROLE group_read_write;

-- 3. Attribution des privilèges au groupe LECTURE SEULE

GRANT CONNECT ON DATABASE ma_base TO group_read_only;

GRANT USAGE ON SCHEMA public TO group_read_only;

GRANT SELECT ON ALL TABLES IN SCHEMA public TO group_read_only;

GRANT SELECT ON ALL SEQUENCES IN SCHEMA public TO group_read_only;

-- 4. Attribution des privilèges au groupe LECTURE-ÉCRITURE

-- Le groupe RW hérite des droits du groupe RO (il peut lire)

GRANT group_read_only TO group_read_write;

-- On ajoute les droits de modification

GRANT INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO group_read_write;

GRANT USAGE, UPDATE ON ALL SEQUENCES IN SCHEMA public TO group_read_write;

-- 5. Automatisation pour les futures tables

-- Pour que les nouvelles tables soient accessibles sans relancer le script

ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT SELECT ON TABLES TO
group_read_only;

ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT INSERT, UPDATE, DELETE ON TABLES
TO group_read_write;

ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT USAGE, UPDATE ON SEQUENCES TO
group_read_write;

-- 6. Création des utilisateurs finaux et affectation

CREATE USER user_consultant WITH LOGIN PASSWORD 'pass_lecture';

CREATE USER user_analyste WITH LOGIN PASSWORD 'pass_ecriture';

GRANT group_read_only TO user_consultant;

GRANT group_read_write TO user_analyste;
```